

Implementation and Performance Analysis of GCD Using Euclid's Algorithm

Student Name: Ajaya Kumar Patel

Roll Number: CRS2503

Course: Cryptographic and Security Implementations

July 29, 2026

Contents

1	Introduction	2
2	Euclid's Algorithm	2
2.1	Algorithm	2
3	C Program	3
4	Dry Run	4
5	Complexity Analysis	5
6	Benchmark Methodology	5
7	Benchmark Results	6
8	Conclusion	6
9	References	7

1 Introduction

The Greatest Common Divisor (GCD) of two integers is the largest integer that divides both numbers without leaving a remainder. Euclid's Algorithm provides an efficient method for computing GCD using repeated modulo operations. This report presents the implementation of Euclid's Algorithm in C and evaluates its execution performance using time and CPU cycle measurements.

2 Euclid's Algorithm

Euclid's Algorithm is based on the following mathematical property:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

The algorithm repeatedly replaces the pair (a, b) with

$$(b, a \bmod b)$$

until the second number becomes zero. The remaining value of a is the Greatest Common Divisor.

2.1 Algorithm

1. Read two positive integers a and b .
2. Repeat the following steps while:

$$b \neq 0$$

- (a) Compute the remainder:

$$r = a \bmod b$$

- (b) Assign:

$$a = b$$

- (c) Assign:

$$b = r$$

3. Return a as the GCD.

3 C Program

Listing 1: GCD Calculation Using Euclidean Algorithm

```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include <time.h>
4 #include <x86intrin.h>
5
6 #define ITERATIONS 1000000ULL
7
8 void benchmark_gcd(unsigned int a, unsigned int b);
9 unsigned int gcd(unsigned int a, unsigned int b);
10
11 int main(){
12     unsigned int a, b;
13
14     printf("Enter two positive integers: ");
15     scanf("%u %u", &a, &b);
16
17     unsigned int result = gcd(a, b);
18
19     printf("GCD = %u\n", result);
20
21     // Time and cycle analysis
22     benchmark_gcd(a, b);
23
24     return 0;
25 }
26
27 unsigned int gcd(unsigned int a, unsigned int b){
28     while (b != 0)
29     {
30         unsigned int r = a % b;
31         a = b;
32         b = r;
33     }
34     return a;
35 }
36
37
38 void benchmark_gcd(unsigned int a, unsigned int b)
39 {
40     clock_t start_time, end_time;
41
42     unsigned int aux;
```

```

43
44     uint64_t start_cycles, end_cycles;
45
46     volatile unsigned int result = 0;
47
48     start_time = clock();
49     start_cycles = __rdtscp(&aux);
50
51     for(uint64_t i = 0; i < ITERATIONS; i++)
52     {
53         result = gcd(a, b);
54     }
55
56     end_cycles = __rdtscp(&aux);
57     end_time = clock();
58
59     double total_time =
60         ((double)(end_time - start_time)) / CLOCKS_PER_SEC;
61
62     double average_time =
63         total_time / ITERATIONS;
64
65     double average_cycles =
66         (double)(end_cycles - start_cycles) / ITERATIONS;
67
68
69     printf("\n===== Benchmark =====\n");
70     printf("Iterations           : %llu\n", ITERATIONS);
71     printf("Total Time               : %.6f seconds\n", total_time);
72     printf("Average Time              : %.12f seconds\n", average_time);
73     printf("Average CPU Cycles       : %.2f cycles\n", average_cycles);
74 }

```

4 Dry Run

Consider the following input:

$$a = 48, \quad b = 18$$

The execution of Euclid's Algorithm is shown below:

Since the remainder becomes zero in the third iteration, the algorithm stops. The value of a at this point is the Greatest Common Divisor.

$\text{gcd}(48, 18) = 6$

Iteration	a	b	$a \bmod b$
1	48	18	12
2	18	12	6
3	12	6	0

Table 1: Dry Run of Euclid’s Algorithm

5 Complexity Analysis

In cryptography, the input size is measured by the **number of bits** required to represent the input integers rather than their numerical values.

Let the input integers be represented using n bits. Therefore,

$$2^{n-1} \leq a, b < 2^n$$

The number of iterations performed by Euclid’s Algorithm is proportional to the bit length of the input. In the worst case, the algorithm requires a linear number of iterations with respect to the input size.

The Euclidean Algorithm requires at most $O(n)$ iterations for n -bit input integers. Therefore, the iteration complexity is:

$$O(n)$$

The algorithm uses only a fixed number of variables, therefore the space complexity is:

$$O(1)$$

6 Benchmark Methodology

The execution performance of the GCD implementation was measured by executing the algorithm multiple times instead of a single execution, because a single execution completes too quickly for accurate timing.

The benchmark was performed using:

- Number of iterations: 1,000,000
- Timer: C standard library `clock()`
- CPU cycle measurement: `_rdtscp()` instruction
- Compiler optimization levels: `-O0` and `-O3`

The average execution time and average CPU cycles per GCD operation were calculated using:

$$\text{Average Time} = \frac{\text{Total Execution Time}}{\text{Number of Iterations}}$$

$$\text{Average Cycles} = \frac{\text{Total CPU Cycles}}{\text{Number of Iterations}}$$

7 Benchmark Results

Input values:

$$a = 123456789, \quad b = 987654321$$

The obtained GCD value is:

$$\text{gcd}(123456789, 987654321) = 9$$

The benchmark results are shown below.

Optimization	Average Time	Average CPU Cycles
-O0	15.852 ns	39.56 cycles
-O3	9.860 ns	24.59 cycles

Table 2: GCD Benchmark Results

8 Conclusion

The Euclidean Algorithm was successfully implemented in C for computing the Greatest Common Divisor of two integers. The implementation was validated using different inputs and its execution performance was measured using repeated iterations.

The benchmark results show that compiler optimization significantly improves execution performance. The `-O3` optimization level reduced both execution time and CPU cycle count compared to the unoptimized version.

9 References

1. GeeksforGeeks, “Euclidean Algorithms (Basic and Extended),” Available: <https://www.geeksforgeeks.org/euclidean-algorithms-basic-and-extended/>. Accessed: July 22, 2026.
2. OpenAI, *ChatGPT (GPT-5.5)*, Large Language Model. Available: <https://chatgpt.com/>. Accessed: July 22, 2026.